Qwen

From Intent to Art: A Hybrid Computation Strategy for Real-Time Safety and Asynchronous Research in PromptLang-Lua

Architectural Blueprint for Purity and Extensibility

The development of an extensible art-behavior metric system for the PromptLang-Lua framework necessitates a robust architectural blueprint that prioritizes purity, allocation-light execution, and future adaptability . This approach ensures that the core logic remains predictable and testable while providing a scalable foundation for the addition of new metrics without compromising performance or stability. The proposed architecture leverages the distinct strengths of two complementary languages: Lua for its agility in scripting, routing, and high-level policy application, and Rust for its performance and memory safety in computationally intensive backend processing [30]. This dual-language strategy creates a layered system where Lua acts as the dynamic front-end, interacting directly with the user and orchestrating the overall flow, while Rust provides the heavy-lifting computational engine for the metrics themselves [30].

The principle of maintaining pure, side-effect-free functions is central to this design. In both Lua and Rust, each metric computation is encapsulated within a function that takes clearly defined input parameters and returns a deterministic numeric score [6]. This functional paradigm, common in languages like Lua, ensures that the output of a metric calculation is solely dependent on its inputs, making it easier to reason about, debug, and unit test [6]. For instance, the Lua module `modules/metrics.lua` defines a set of standalone functions like `M.compute_edf` and `M.compute_nsg`, each designed to operate independently on a structured input table . Similarly, the Rust implementation in `src/metrics_ext.rs` defines a series of methods within the `ArtBehaviorMetrics` struct that perform calculations based on provided input structs like `AnalyzerScores` and `SanitizerSummary` . This separation of concerns allows the business logic of each metric to evolve independently. Adding a new metric, such as a hypothetical `Emotional Contagion Index (ECI)`, would involve creating a new function in the Lua module and a corresponding field in the Rust struct, without needing to modify the existing

metric implementations [47]. This modularity is the cornerstone of achieving long-term extensibility.

Performance considerations are addressed through the commitment to allocation-light functions, a critical requirement for systems that may need to compute these metrics on a per-prompt basis in real-time . While aggressive micro-optimizations are deferred in favor of maintainability, the current design already avoids complex allocations within the hot path of metric computation . The functions operate primarily on scalar values and simple data structures, minimizing memory churn. This is particularly important in the context of garbage-collected languages like Lua, where excessive object creation can lead to performance degradation [7] [22]. By designing the functions to be stateless and work with primitive types, they become ideal candidates for inlining and optimization at a later stage if performance bottlenecks are identified [6]. The use of Rust for the backend further solidifies this commitment, as Rust's ownership model and lack of a garbage collector provide fine-grained control over memory, ensuring that even complex computations can be performed with predictable performance characteristics [7]. The provided Rust code exemplifies this by defining structs like `AnalyzerScores` and `SanitizerSummary` that hold primitive numerical data, facilitating efficient serialization, deserialization, and computation .

The integration with the existing `VirtualMetrics` system is a key aspect of the architecture, serving as the nexus where these new metrics are stored, updated, and exposed to the rest of the system . The `VirtualMetrics` system, which likely tracks core indicators like Intent-to-Expressiveness Index (IEI) and Transformable Intent Score (TIS), is extended to include the new `ArtBehaviorMetrics` struct . This can be achieved by adding a field of type `ArtBehaviorMetrics` to the `VirtualObject` definition or by maintaining a separate map keyed by `VirtualNodeId`. During the main computational pass, after initial intent analysis and sanitization, the `MetricEngine` (the Rust component) would call the `ArtBehaviorMetrics::compute` function, passing in all necessary data from analyzers, sanitizers, and event logs . The resulting `ArtBehaviorMetrics` instance is then attached to the `VirtualObject` being processed. This instance is subsequently serialized into the JSON snapshot that is sent to the Lua frontend, making all ten metric scores available for routing decisions, scripted-learning algorithms, and game-like interactions . This tight coupling ensures that the metrics are always up-to-date and directly influence the behavior of the virtual environment. The entire system is framed within a larger simulation context, where each virtual object can be viewed as a district in a colony, and the metrics describe the health, volatility, and cohesion of that district over time .

Core Metric Formulations and Non-Linear Behaviors

The implementation of the ten art-behavior metrics requires careful formulation of their mathematical definitions, with specific enhancements to introduce non-linear responses as requested. These metrics are not isolated calculations but form a coherent system for quantifying the transformation of user intent from potentially destructive states into constructive or artistic ones. The foundational metrics, Expressive Destruction Flux (EDF) and Abstract Fireball Index (AFI), are central to this transformation, directly building upon the user's initial conceptual formula $E+D-UI=A^2$. EDF is defined as the normalized product of emotional intensity and destructive intent, signaling the raw energy available for artistic conversion . AIFI is a specialized case of TIS focused on prompts suitable for becoming abstract, high-energy scenes like swirling colors or fireballs . To fulfill the directive for non-linear behavior, these two metrics must be modified to apply a soft boost when emotional intensity is high. Specifically, when the emotional intensity score is greater than or equal to 0.7, the contribution of this component to the final score should be amplified, for example, by squaring it. This introduces a threshold effect, making the system disproportionately sensitive to highly emotional prompts and pushing them more strongly toward abstract artistic modes of expression. This aligns with the goal of containing and redirecting intense emotional content before it can escalate.

A second critical non-linear behavior is the implementation of hard saturation to prevent unsafe prompt encouragement. The directive specifies that if any single danger channel—such as violence, self-harm, or hate—is scored at or above 0.9, the system must not produce a misleadingly low safety score, regardless of other mitigating factors . This acts as a crucial safety guardrail. In the implementation, this is best handled as an early-exit condition in the primary computation logic, likely within the Rust backend for performance and safety reasons. Before any complex aggregation occurs, the system should check the danger levels from the original prompt's analyzer scores. If any danger channel exceeds the 0.9 threshold, the metric computation should immediately return a low score indicative of a high-risk state, forcing the system to prioritize containment over creative redirection. This prevents the model from ever inadvertently encouraging borderline unsafe content by allowing one extremely dangerous signal to be "drowned out" by positive transformations in other areas. This hard constraint is essential for maintaining the integrity of the safety framework.

The remaining eight metrics focus on evaluating different facets of the sanitization and redirection process, providing a comprehensive view of its quality and effectiveness. | Metric Name | Acronym | Description | Key Inputs | | :--- | :--- | :--- | :--- | | Harm-to-Color Divergence | HCD | Measures the conversion of lexical harm into color, motion, or

metaphorical language. | Sanitizer output (replaced tokens, metaphor tokens) | | Narrative Safety Gradient | NSG | Tracks the change in danger score between the original and sanitized prompt. | Analyzer scores (before/after) | | Scene Fragmentation Ratio | SFR | Fraction of a prompt that can be split into independent, non-violent visual chunks. | Sanitizer output (safe segments, total segments) | | Calm Recomposition Score | CRS | Measures movement towards calm descriptors while preserving expressive amplitude. | Sanitizer output (calm tokens, impact tokens) | | Metaphor Saturation Coefficient | MSC | Quantifies the density of metaphorical tokens versus explicit harmful terms. | Sanitizer output (metaphor tokens, trigger tokens) | | Redirected Intent Yield | RIY | Proportion of flagged prompts successfully redirected into art modules instead of being denied. | Session history (flagged, redirected prompts) | | Expressive Safety Envelope | ESE | Per-user/session cap on destructive amplitude that can be safely transformed. | Analyzer scores, session events, history, age band | | Colony Pattern Harmony | CPH | Measures how well virtual objects resemble a stable colony/settlement pattern. | Node-level metrics (IEI, STS, PKC) |

These metrics provide granular feedback for both immediate system tuning and long-term research. HCD, NSG, CRS, and MSC evaluate the quality of the sanitization rewrite itself, ensuring that harm is genuinely de-escalated and artistically transformed . They answer questions about whether metaphors are being used effectively (MSC), if the rewrite truly makes the prompt calmer (CRS), and if the overall danger level has been reduced (NSG). These metrics are ideal for asynchronous computation in a research lab, where their collective output can be analyzed to refine the sanitization algorithms and training data . On the other hand, RIY and CPH offer macroscopic views of system performance. RIY measures the overall success rate of the redirection policy, providing a key performance indicator for the safety framework . CPH provides a holistic assessment of the virtual environment's health, determining if interactions are coalescing into stable, constructive patterns or devolving into chaotic conflict . The formulas for these metrics, as implemented in the provided Lua and Rust code, combine various weighted inputs to produce a normalized score between 0 and 1, ready for consumption by downstream systems .

The Expressive Safety Envelope and Age-Band Clamping

The Expressive Safety Envelope (ESE) stands as a pivotal metric, uniquely positioned to translate high-level safety policies into personalized, real-time constraints for individual users. Its primary function is to act as a per-session scalar that caps the amount of

destructive amplitude that can be transformed into art, thereby preventing the system from soft-blocking or heavily abstractifying user content . The implementation of ESE is a direct response to the need for a nuanced, adaptive safety model that goes beyond static filters. It achieves this by synthesizing multiple streams of information: the user's raw destructive intent from the analyzer, the interactive nature of the session as captured by event logs, the user's safety history over multiple sessions, and most critically, the user's designated age band . This multi-faceted approach ensures that safety enforcement is not monolithic but is calibrated to the perceived maturity and vulnerability of the user.

The core of the ESE's functionality lies in its ability to enforce strict, age-band-aware clamps. The provided Lua implementation correctly models this requirement, applying distinct scaling factors based on the user's age group . For a user classified as a "CHILD," the destructive component of their intent has its effective amplitude scaled down by a factor of 0.25. For a "YOUNGTEEN," the factor is 0.4, rising to 0.6 for a "TEEN," and only reaching its full value of 1.0 for an "ADULT" . This hierarchical scaling is a powerful safety mechanism, ensuring that younger users are protected from experiencing even a fraction of the potentially harmful destructive content they might have initiated, regardless of what the underlying analyzer scores suggest [2] [3] . This directly addresses a significant challenge highlighted in LLM safety research, where models often exhibit systematic weaknesses in interacting with children and may overfit to adult-normative interaction patterns [4] . The ESE acts as a corrective filter, grounding the system's behavior in developmental psychology principles rather than relying solely on algorithmic signals [1] .

The theoretical underpinnings for such age-based differentiation are supported by benchmarks like ChildSafe, which evaluates LLM safety using simulated agents representing different developmental stages [1] [5] . These benchmarks demonstrate that safety priorities shift significantly with age; for example, 'Emotional Safety' and 'Privacy Protection' are given much higher weights for younger children (ages 6-8), whereas 'Boundary Respect' and 'Manipulation Resistance' become more salient for adolescents (ages 15-17) [4] . The ESE's fixed, pre-programmed clamps represent a simplified but effective version of this principle. While more sophisticated implementations could dynamically adjust weights based on a richer profile, the current approach provides a robust and easily implementable safeguard. The successful operation of this feature hinges on the accurate and consistent determination of the `age_band` string upstream in the system before it is passed to the `compute_ese` function . Any error in this classification would cascade through the safety logic, making the reliability of this input paramount.

Beyond the age-band clamp, the ESE incorporates additional layers of safety logic. It considers the user's recent interactive history, represented by the `total_events` count, which scales the envelope up or down based on engagement density . A user who is rapidly firing off a sequence of high-intent prompts might see their envelope shrink more quickly than a user who is more deliberate in their interactions. Furthermore, the ESE integrates long-term history, using the ratio of `floodrouted_prompts` to `total_prompts` to gauge the user's overall tendency to trigger safety interventions . A user with a history of frequent flood-routing would receive a lower safety envelope, reflecting a learned risk profile. The final score is a carefully balanced equation, combining these elements to produce a fluid, context-aware safety ceiling. This sophisticated interplay of real-time intent, session dynamics, historical behavior, and user demographics makes the ESE a cornerstone of the system's adaptive safety posture, ensuring that artistic expression is encouraged within safe and appropriate boundaries for every user.

Strategic Integration with VirtualMetrics and Hybrid Computation

The seamless integration of the ten art-behavior metrics into the existing `VirtualMetrics` system and the strategic adoption of a hybrid synchronous/asynchronous computation model are fundamental to the operational success of the PromptLang-Lua framework. This approach directly addresses the dual needs of real-time decision-making and long-term analytical research, ensuring that the system is both responsive and continuously improving. The `VirtualMetrics` system serves as the central repository for all quantitative data about the state of a `VirtualObject`—be it a chat panel, a canvas, or a game element. Integrating the new `ArtBehaviorMetrics` struct into this system means extending the object's state to include these ten new scores, making them accessible wherever the object's status is queried . In the Rust backend, this involves modifying the `VirtualObject` struct to contain an `ArtBehaviorMetrics` field. After the primary analysis and sanitization pipeline runs, the `MetricEngine` computes the full suite of ten metrics and populates this struct. This enriched object state is then passed to the Lua layer, typically via a JSON serialization of the `VirtualObject` snapshot, exposing all ten metric scores as first-class properties on the object's metatable . This tight integration ensures that routing scripts, policy matrices, and game logic can react instantly to the nuanced state of the system.

The distinction between synchronous and asynchronous computation is a critical strategic decision that optimizes resource usage and aligns metric utility with its purpose. The user's directive mandates that four specific metrics—Expressive Destruction Flux (EDF), Narrative Safety Gradient (NSG), Abstract Fireball Index (AFI), and Expressive Safety Envelope (ESE)—be computed synchronously during the prompt sanitization and routing path . These metrics are safety-critical and must inform live decisions with minimal latency. A high `AFI` score, for instance, might immediately trigger a switch to a high-energy painting mode, while a low `ESE` score for a child user could force a soft-block or a pivot to a gentler interaction style . Because their results directly shape the user's immediate experience, these computations must be allocation-light and fast. The remaining six metrics—Harm-to-Color Divergence (HCD), Scene Fragmentation Ratio (SFR), Calm Recomposition Score (CRS), Metaphor Saturation Coefficient (MSC), Redirected Intent Yield (RIY), and Colony Pattern Harmony (CPH)—are intended for asynchronous computation in a background "lab" process . Their purpose is research, logging, and future model tuning, so they can afford to be computationally more intensive. For example, calculating `Colony Pattern Harmony (CPH)` might require analyzing graph connectivity across thousands of nodes, a task far better suited for a batch job running on historical profiling logs rather than incurring the cost on every user interaction .

This hybrid model necessitates a clear separation of concerns in the system's execution pipeline. A dedicated, lightweight module would be responsible for calling the four synchronous metric functions inline during the main request-response cycle. This module would feed the minimal required inputs (analyzer scores, basic sanitizer data) and make immediate routing decisions based on the returned scores. Simultaneously, a separate "Research Lab" service would consume the full event and profiling logs generated by the system. This lab service would replay sessions, re-run the more expensive metric calculations, and store the results in a data warehouse for analysis [28] [38]. The output of this lab, particularly the `RIY` metric, becomes invaluable for tuning the weighting parameters in all metric formulas, creating a feedback loop for continuous improvement . This division allows the production system to remain lean and performant while still generating rich, detailed data for deep analytical study. It embodies a modern software architecture practice of separating real-time data processing from offline analytics, ensuring that the system can scale to serve many users in real-time without sacrificing the depth of its analytical capabilities [10] [11]. The result is a system that is not only reactive but also proactively learning and evolving based on its own behavioral data.

Data Modeling and Input Requirements for Metric Computation

The accuracy and consistency of the art-behavior metrics depend entirely on the quality and structure of the data provided to their computation functions. The system requires a well-defined set of input data models to capture the necessary information from the prompt analysis, sanitization, and event logging phases. The provided Lua and Rust implementations already define these contracts, specifying the exact tables and structs required by each metric function . A robust implementation must ensure that the upstream components of the system—namely, the analyzer, the sanitizer, and the event logger—all adhere to this contract precisely. This data modeling forms the bedrock of the entire metrics system, and any discrepancies or omissions in the data supply will lead to inaccurate or unreliable metric scores.

For the Lua-side `modules/metrics.lua`, the required inputs are defined as several optional tables, each containing a specific set of fields. The `analyzer` table, for instance, must contain keys like `violence`, `selfharm`, `adult`, `emotional`, and others, with each value being a number between 0 and 1 . These scores are typically generated by a machine learning model that scans the prompt text. The `sanitizer` table contains the results of the rewriting process, with fields such as `original_text`, `sanitized_text`, `trigger_tokens`, `replaced_tokens`, `metaphor_tokens`, `calm_tokens_added`, and `impact_tokens_removed` . The logic for populating these token counts is a critical part of the sanitization process and must be consistently defined. For example, `replaced_tokens` would be the count of words replaced with safer alternatives, while `metaphor_tokens` would be the count of words replaced with metaphorical constructs. The `events` table summarizes activity related to a node or session, with fields like `total_events`, `distinct_kinds`, and `last_timestamp_ms` . Finally, the `history` table holds longer-term statistics for a user or session, including `total_prompts`, `redirected_prompts`, and `floodrouted_prompts` . The `age_band` is a string ("CHILD", "YOUNGTEEN", etc.), and `node_metrics` contains additional per-node data like IEI and STS .

The Rust implementation mirrors this structure with strongly-typed structs, providing an even stricter contract. The `AnalyzerScores` struct enforces the presence of fields like `pub violence: f32`, `pub emotional: f32`, etc. . Similarly, `SanitizerSummary` defines `pub trigger_tokens: u32` and `pub metaphor_tokens: u32` . This rigid typing in Rust helps catch errors at compile time that might otherwise go unnoticed in a dynamically-typed language like Lua. The parallel between the Lua tables and Rust structs ensures that the data can be seamlessly passed between the two components.

When the Rust backend processes an event, it receives a JSON payload from the Lua side or reads directly from a shared memory structure. It deserializes this data into the corresponding Rust structs, performs the calculations, and then serializes the resulting `ArtBehaviorMetrics` struct back into a format that the Lua frontend can understand and attach to a `VirtualObject`.

The table below outlines the complete data model required for the ten metrics, mapping each metric to its specific input dependencies and summarizing the source of the required data.

Metric Name	Dependent Input Fields	Source Component
EDF	`analyzer.emotional`, `analyzer.violence`, `analyzer.selfharm`	Analyzer
NSG	`analyzer_before` (all danger channels), `analyzer_after` (all danger channels)	Analyzer
AFI	`analyzer.violence`, `analyzer.selfharm`, `analyzer.motion`, `analyzer.emotional`	Analyzer
ESE	`analyzer.violence`, `analyzer.selfharm`, `analyzer.emotional`, `events`, `history`, `age_band`	Analyzer, Event Logger, History Store
HCD	`sanitizer.replaced_tokens`, `sanitizer.trigger_tokens`, `sanitizer.metaphor_tokens`, `sanitizer.calm_tokens_added`	Sanitizer
SFR	`sanitizer.safe_segments`, `sanitizer.total_segments`	Sanitizer
CRS	`sanitizer.calm_tokens_added`, `sanitizer.impact_tokens_removed`, `analyzer_before.emotional`, `analyzer_after.emotional`	Sanitizer, Analyzer
MSC	`sanitizer.metaphor_tokens`, `sanitizer.trigger_tokens`	Sanitizer
RIY	`history.flagged_prompts`, `history.redirected_prompts`	History Store
CPH	`node_metrics.iei`, `node_metrics.tis`, `node_metrics.sts`, `node_metrics.pkc`	Node State Manager

This structured data flow is essential for the system's operation. The `Analyzer` component must reliably scan prompts and output standardized scores. The `Sanitizer` must meticulously track its rewriting actions and expose these counts. The `Event Logger` and `History Store` must maintain accurate session and user-level statistics. The `Node State Manager` must propagate the extended metrics to all relevant parts of the system. By formalizing this data contract, the system ensures that the ten art-behavior metrics receive the precise information they need to compute their scores accurately, forming a reliable basis for all subsequent routing and policy decisions.

Synthesis and Strategic Recommendations for Implementation

This research report has detailed the comprehensive plan for implementing ten art-behavior metrics within the PromptLang-Lua framework, fulfilling the core research goal of creating a system that balances artistic expression with adaptive safety. The proposed solution is built upon a sound architectural blueprint emphasizing purity, extensibility, and allocation-light functions in both Lua and Rust . The system leverages a hybrid computation model, separating four metrics (EDF, NSG, AFI, ESE) for real-time, synchronous decision-making from six others (HCD, SFR, CRS, MSC, RIY, CPH) for asynchronous, research-focused analysis . This design directly supports the project's overarching objective of transforming potentially harmful user inputs into constructive artistic outputs while enabling deep, data-driven refinement of the system's behavior over time.

The implementation of non-linear behaviors is a critical enhancement that elevates the system from a simple linear classifier to a more nuanced and context-aware safety engine. The introduction of a soft boost for high emotional intensity ($\geqslant 0.7$) in EDF and AFI ensures the system is more sensitive to prompts carrying strong emotional weight, pushing them more decisively toward abstract artistic forms . Concurrently, the hard saturation guardrail, which prevents metric scores from encouraging unsafe content when any danger channel is $\geqslant 0.9$, provides an indispensable failsafe against misinterpretation or adversarial manipulation of the system's logic . The `Expressive Safety Envelope (ESE)` metric serves as a powerful embodiment of this adaptive safety philosophy, incorporating strict, age-band-aware clamps that personalize the safety threshold for each user, particularly protecting younger audiences from exposure to destructive content . This personalization is a significant step forward in creating inclusive and safe AI interactions.

To move from this conceptual and code-ready state to a fully operational system, several strategic recommendations are proposed. First, the core metric logic in both the Lua and Rust files must be updated to incorporate the specified non-linear behaviors. The emotional intensity component in `compute_edf` and `compute_afi` should be modified to apply a squaring or similar non-linear function when the score is 0.7 or higher. A hard saturation check must be added to the beginning of the Rust `ArtBehaviorMetrics::compute` function to early-exit with a conservative safety score if any danger channel from the original prompt exceeds the 0.9 threshold. Second, the system's execution pipeline must be refactored to support the hybrid computation model. A dedicated, low-latency module should be created for the four synchronous

metrics, ensuring they run inline during the prompt sanitization path. Separately, a "Research Lab" infrastructure should be developed to process historical profiling logs and compute the remaining six asynchronous metrics in a batch fashion. Third, a formal protocol for data exchange must be established and enforced. The `Analyzer`, `Sanitizer`, `Event Logger`, and `History Store` components must be guaranteed to populate their respective data structures according to the contracts defined in the metric functions, ensuring data integrity and consistency. Finally, a robust evaluation framework must be developed to validate the system's performance. Drawing inspiration from academic benchmarks like ChildSafe, the team should create simulation agents representing different age groups and cognitive styles to stress-test the system's safety, responsiveness, and expressiveness under controlled conditions [1] [4]. The data generated by these tests, combined with the logged asynchronous metrics, will provide the empirical basis for iteratively tuning the system's parameters and driving its continuous evolution. By following these steps, the PromptLang-Lua framework can be realized as a sophisticated, safe, and artistically expressive AI system capable of navigating the complex interplay between human emotion, digital safety, and creative potential.

Reference

1. Evaluating LLM Safety Across Child Development Stages https://arxiv.org/html/2510.05484v1
2. (PDF) LLM Safety for Children https://www.researchgate.net/publication/389130354_LLM_Safety_for_Children
3. A Developmental Benchmark for Evaluating LLM Safety in ... https://arxiv.org/abs/2506.13510
4. Evaluating LLM Safety Across Child Development Stages https://arxiv.org/html/2510.05484v2
5. Evaluating LLM Safety Across Child Development Stages https://www.researchgate.net/publication/396291343_Evaluating_LLM_Safety_Across_Child_Development_Stages_A_Simulated_Agent_Approach
6. Is Lua based primarily on well-established programming ... https://stackoverflow.com/questions/2100902/is-lua-based-primarily-on-well-established-programming-language-ideas

7. To GC or Not To GC - c++ https://stackoverflow.com/questions/7575055/to-gc-or-not-to-gc
8. Information Systems Development https://www.researchgate.net/profile/Dijana-Plantak-Vukovac/publication/271586132_Information_Systems_Development_Transforming_Organisations_and_Society_Through_Information_Systems/links/630376b8ceb9764f7216ca33/Information-Systems-Development-Transforming-Organisations-and-Society-Through-Information-Systems.pdf
9. https://github.com/avelino/awesome-go 清单 - unionline https://www.cnblogs.com/fanbi/p/12889385.html
10. Sync-Async without ccBPM https://help.sap.com/docs/SUPPORT_CONTENT/xi/3362977337.html
11. PSME REST API Specification https://www.intel.com/content/dam/www/public/us/en/documents/guides/psme-api-spec-v2-2.pdf
12. 9 - Introscope Workstation User Guide https://www.scribd.com/document/270141207/9-Introscope-Workstation-User-Guide
13. (PDF) Integrating Quality Assurance Techniques with ... https://www.academia.edu/3028397/Integrating_Quality_Assurance_Techniques_with_Requirements_Negotiation_Activities_An_Analysis_of_Software_Developers_Practices
14. State-of-the-Art Review on the Flexural Behavior of Carbon ... https://www.sciencedirect.com/science/article/pii/S135983682600065X
15. A Review of Nonlinear Mechanisms for Frequency Up- ... https://www.mdpi.com/2076-0825/12/12/456
16. Nonisolated High Step-Up DC–DC Converters https://ieeexplore.ieee.org/iel7/63/10339150/10091160.pdf
17. The role of length scales in material failure https://hal.science/tel-04981472v1/file/HDR_GMolnar.pdf
18. Translational Insights from Rodent and Human Studies - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC12564412/
19. Advancing digital healthcare engineering for aging ships ... https://www.cambridge.org/core/journals/data-centric-engineering/article/advancing-digital-healthcare-engineering-for-aging-ships-and-offshore-structures-an-indepth-review-and-feasibility-analysis/F9FA754BEE7E2F1679D343240E16B843
20. Findings of the Association for Computational Linguistics https://aclanthology.org/volumes/2025.findings-acl/
21. Automatic non-functional testing and tuning of configurable ... https://theses.hal.science/tel-01619139/file/BOUSSAA_Mohamed.pdf

22. Simple Index https://mirrors.sustech.edu.cn/pypi/simple/
23. Sensors, Volume 25, Issue 19 (October-1 2025) https://www.mdpi.com/1424-8220/25/19
24. Proceedings of the 4th International Conference on Electronic ... https://link.springer.com/content/pdf/10.1007/978-981-96-0644-3.pdf
25. A NOISE-ASSISTED DATA ANALYSIS METHOD https://doi.org/10.1142/S1793536909000047?urlappend=%3Futm_source%3Dresearchgate.net%26utm_medium%3Darticle
26. The Application of Biomaterials for the Vaccine, Treatment ... https://pubs.acs.org/doi/10.1021/acsomega.3c08326
27. Design and experimental investigation on VL-MLI intended ... https://www.researchgate.net/publication/357669584_Design_and_experimental_investigation_on_VL-MLI_intended_for_half_height_H-H_method_to_improve_power_quality_using_modified_particle_swarm_optimization_MPSO_algorithm
28. Model Learning for Performance Prediction of Cloud-native ... https://www.researchgate.net/publication/359558208_Model_Learning_for_Performance_Prediction_of_Cloud-native_Microservice_Applications/fulltext/624355917931cc7ccf035196/Model-Learning-for-Performance-Prediction-of-Cloud-native-Microservice-Applications.pdf
29. What's the most efficient way to ignore code in lua? https://stackoverflow.com/questions/11720948/whats-the-most-efficient-way-to-ignore-code-in-lua
30. Complete and Quick Guide to Lua 🌑 https://dev.to/dmitrevnik/the-most-complete-and-quick-guide-to-lua-1k2n
31. Conference Proceedings https://ieeexplore.ieee.org/iel7/6708576/6718436/06718441.pdf
32. ABSTRACTS https://www.jstor.org/stable/pdf/44433839.pdf
33. Methodologies for the Design, the Modeling, and ... https://theses.hal.science/tel-04869563v1/file/VINAGREROGUTIERREZ_2024_diffusion.pdf
34. Calculate value based on amount of green against blue ... https://stackoverflow.com/questions/43082494/calculate-value-based-on-amount-of-green-against-blue-and-red
35. Amazon Linux Security Center - CVE List https://explore.alas.aws.amazon.com/
36. Simple Index https://mirror.nju.edu.cn/pypi/web/simple
37. Computer Science https://www.arxiv.org/list/cs/new?skip=25&show=1000
38. AWS re:Invent 2019 Sessions & Podcast Feed http://aws-reinvent-audio.s3-website.us-east-2.amazonaws.com/2019/2019.html

39. Lawrence Berkeley National Laboratory https://escholarship.org/content/qt14k6b2rs/qt14k6b2rs.pdf
40. Energy Aware Edge Computing A Survey | PDF https://www.scribd.com/document/691426917/Energy-aware-edge-computing-A-survey
41. Centreon en | PDF | Php | Databases https://www.scribd.com/document/661324256/Centreon-En
42. glove.6B.100d.txt-vocab.txt https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt
43. Lucene Change Log https://lucene.apache.org/core/5_5_0/changes/Changes.html
44. Genetics and Neurodegenerative Diseases | PDF https://www.scribd.com/document/998988178/Genetics-and-Neurodegenerative-Diseases
45. Endocrinology Meet Teacher https://pdfcoffee.com/endocrinology-meet-teacher-pdf-free.html
46. 333333 23135851162 the 13151942776 of 12997637966 https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
47. Rust define trait interface for any type? https://stackoverflow.com/questions/73412151/rust-define-trait-interface-for-any-type
48. Regular versus Irregular Exercise Differentially Modulates ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12677590/
49. Spatiotemporal cerebral blood flow dynamics underlies ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC11030426/
50. Multi-omics insight into muscle quality divergence between ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12634325/
51. Evidence from Animal, Clinical, and Multi-Omics Studies - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC12986025/
52. Divergent Cell-Type Specific Hypoxia Responses in Human ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12330707/
53. Selective Modulation of PAR-2-Driven Inflammatory Pathways ... https://pmc.ncbi.nlm.nih.gov/articles/PMC11988659/
54. Divergent Molecular and Cellular Responses to Low and High ... https://pmc.ncbi.nlm.nih.gov/articles/PMC9739411/
55. Brain Photobiomodulation Therapy: A Narrative Review - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC6041198/
56. Pool-Fire-and-Fireball-Experiments-in-Support-of-the-US- ... https://www.researchgate.net/profile/Joseph-Hogge/publication/335797132_Pool_Fire_and_Fireball_Experiments_in_Support_of_the_US_DOEDOTTC

_Crude_Oil_Characterization_Research_Study/links/5d7bc604a6fdcc2f0f6d8597/Pool-Fire-and-Fireball-Experiments-in-Support-of-the-US-DOE-DOT-TC-Crude-Oil-Characterization-Research-Study.pdf

57. (PDF) Fire Explosion Risk https://www.academia.edu/40861962/Fire_Explosion_Risk
58. Magnox graphite core decommissioning and disposal issues https://www-pub.iaea.org/mtcd/publications/pdf/te_1647_cd/pdf/tecdoc_1647.pdf
59. Harmony COVID-19: A ready-to-use kit, low-cost detector, and ... https://pmc.ncbi.nlm.nih.gov/articles/PMC8673764/
60. A CRISPR/Cas9 screen reveals proteins at the endosome ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12209463/
61. The Impact of SIV-Induced Immunodeficiency on SARS-CoV-2 ... https://pmc.ncbi.nlm.nih.gov/articles/PMC11281476/
62. Single-cell RNA sequencing reveals that MYBL2 in malignant ... https://pmc.ncbi.nlm.nih.gov/articles/PMC11317301/
63. Mycobacterial α-glucans hijack Dectin-1 to facilitate ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC7618678/
64. Abstracts - PMC - NIH https://pmc.ncbi.nlm.nih.gov/articles/PMC7129694/
65. Cosmetics Preservation: A Review on Present Strategies - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC6099538/
66. Questioning Representational Optimism in Deep Learning https://arxiv.org/html/2505.11581v1
67. A Philosophical Introduction to Language Models ... https://arxiv.org/html/2405.03207v1
68. Position Paper: Towards Open Complex Human–AI Agents ... https://arxiv.org/html/2505.00018v1
69. Computer Science Apr 2019 http://arxiv.org/list/cs/2019-04?skip=3500&show=2000
70. Orak: A Foundational Benchmark for Training and ... https://arxiv.org/html/2506.03610v1
71. The Diffusion Duality https://arxiv.org/pdf/2506.10892
72. Energies, Volume 18, Issue 12 (June-2 2025) – 223 articles https://www.mdpi.com/1996-1073/18/12
73. Mechanics for a New Millennium - Springer https://link.springer.com/content/pdf/10.1007/0-306-46956-1.pdf